T2

October 26, 2026

1 Study Project: Adversarial Machine Learning

1.1 Part I: Statistical Analysis and Generation of Process Data

1.2 Group 15

1.3 SIN Wang Yat

1.3.1 Task 2: Similarity between Different Datasets

The goal of this task is to analyze the similarity of the provided physical readings between the normal and malicious physical measurements in the dataset WADI 2017. This code satisfies the following requirements:

- Task 2a Computer the Spearman Distance

Compute the Spearman distance of collected physical readings between every two time steps, existing in the dataset WADI_14days and WADI_attackdata and then differentiate between physical readings containing attack and those without attacks.

- Task 2b Create a Histogram

Creates a single histogram plotting the distribution of the computed Spearman distances for the dataset. The histogram illustrates the distribution of Spearman distances from both normal and malicious physical measurements of WADI_attackdata

1.3.2 Libray Import and Data Loading

[1]:
```
print(f"Start: Import Library and Load the datasets")
# Library Import
import pandas as pd
import numpy as np
from pathlib import Path
import matplotlib.pyplot as plt

# Define the base directory path
base_path = Path.home() / "Projects/P1/data/"
cache_train_file = base_path / "processed/cache_train_file.parquet"
cache_test_file = base_path / "processed/cache_test_file.parquet"

# Load normal 14days data and attack data from CSV or Cache
# Load 14days Dataset
```

```
if cache_train_file.exists():
   print("Loading fast cached Parquet data-14days dataset")
   train17_df = pd.read_parquet(cache_train_file)
else:
   print("Cache not found. Load data from CSV file-14days dataset")
   # Skip 4 meta data rows
   train17_df = pd.read_csv(base_path / "raw/2017/WADI_14days.csv", skiprows=4)
   # Save the cache for next time
   train17_df.to_parquet(cache_train_file)

# Load attackdata Dataset
if cache_test_file.exists():
   print("Loading fast cached Parquet data-attackdata dataset")
   test17_df = pd.read_parquet(cache_test_file)
else:
   print("Cache not found. Load data from CSV file-attackdata dataset")
   test17_df = pd.read_csv(base_path / "raw/2017/WADI_attackdata.csv")
   # Save the cache for next time
   test17_df.to_parquet(cache_test_file)

# Remove the long prefix Column Name
prefix_to_remove = r"\\WIN-25J4RO10SBF\LOG_DATA\SUTD_WADI\LOG_DATA"

# Replace the string in all column names
train17_df.columns = train17_df.columns.str.replace(prefix_to_remove, "",␣
 ↪regex=False)
test17_df.columns = test17_df.columns.str.replace(prefix_to_remove, "",␣
 ↪regex=False)
```

```
Start: Import Library and Load the datasets
Loading fast cached Parquet data-14days dataset
Loading fast cached Parquet data-attackdata dataset
```

[2]:
```
print(f"Data Shape of WADI_14days")
print(train17_df.shape)
```

```
Data Shape of WADI_14days
(1209601, 130)
```

[3]:
```
print(f"Data info of WADI_14days")
print(train17_df.info(verbose=True,show_counts=True))
print(train17_df)
```

```
Data info of WADI_14days
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1209601 entries, 0 to 1209600
Data columns (total 130 columns):
 #    Column                      Non-Null Count    Dtype
```

| --- | ------ | -------------- | ----- |
0	Row	1209601 non-null	int64
1	Date	1209601 non-null	object
2	Time	1209601 non-null	object
3	\1_AIT_001_PV	1209601 non-null	float64
4	\1_AIT_002_PV	1209589 non-null	float64
5	\1_AIT_003_PV	1209601 non-null	float64
6	\1_AIT_004_PV	1209595 non-null	float64
7	\1_AIT_005_PV	1209601 non-null	float64
8	\1_FIT_001_PV	1209601 non-null	float64
9	\1_LS_001_AL	1209601 non-null	int64
10	\1_LS_002_AL	1209601 non-null	int64
11	\1_LT_001_PV	1209601 non-null	float64
12	\1_MV_001_STATUS	1209601 non-null	int64
13	\1_MV_002_STATUS	1209601 non-null	int64
14	\1_MV_003_STATUS	1209601 non-null	int64
15	\1_MV_004_STATUS	1209601 non-null	int64
16	\1_P_001_STATUS	1209601 non-null	int64
17	\1_P_002_STATUS	1209601 non-null	int64
18	\1_P_003_STATUS	1209601 non-null	int64
19	\1_P_004_STATUS	1209601 non-null	int64
20	\1_P_005_STATUS	1209601 non-null	int64
21	\1_P_006_STATUS	1209601 non-null	int64
22	\2_DPIT_001_PV	1209601 non-null	float64
23	\2_FIC_101_CO	1209601 non-null	float64
24	\2_FIC_101_PV	1209601 non-null	float64
25	\2_FIC_101_SP	1209601 non-null	float64
26	\2_FIC_201_CO	1209601 non-null	float64
27	\2_FIC_201_PV	1209601 non-null	float64
28	\2_FIC_201_SP	1209601 non-null	float64
29	\2_FIC_301_CO	1209601 non-null	float64
30	\2_FIC_301_PV	1209601 non-null	float64
31	\2_FIC_301_SP	1209601 non-null	float64
32	\2_FIC_401_CO	1209601 non-null	float64
33	\2_FIC_401_PV	1209601 non-null	float64
34	\2_FIC_401_SP	1209601 non-null	float64
35	\2_FIC_501_CO	1209601 non-null	float64
36	\2_FIC_501_PV	1209601 non-null	float64
37	\2_FIC_501_SP	1209601 non-null	float64
38	\2_FIC_601_CO	1209601 non-null	float64
39	\2_FIC_601_PV	1209601 non-null	float64
40	\2_FIC_601_SP	1209601 non-null	float64
41	\2_FIT_001_PV	1209601 non-null	float64
42	\2_FIT_002_PV	1209601 non-null	float64
43	\2_FIT_003_PV	1209601 non-null	float64
44	\2_FQ_101_PV	1209601 non-null	float64
45	\2_FQ_201_PV	1209601 non-null	float64
46	\2_FQ_301_PV	1209601 non-null	float64

```
47   \2_FQ_401_PV             1209601 non-null  float64
48   \2_FQ_501_PV             1209601 non-null  float64
49   \2_FQ_601_PV             1209601 non-null  float64
50   \2_LS_001_AL             0 non-null        float64
51   \2_LS_002_AL             0 non-null        float64
52   \2_LS_101_AH             1209601 non-null  int64
53   \2_LS_101_AL             1209601 non-null  int64
54   \2_LS_201_AH             1209601 non-null  int64
55   \2_LS_201_AL             1209601 non-null  int64
56   \2_LS_301_AH             1209601 non-null  int64
57   \2_LS_301_AL             1209601 non-null  int64
58   \2_LS_401_AH             1209601 non-null  int64
59   \2_LS_401_AL             1209601 non-null  int64
60   \2_LS_501_AH             1209601 non-null  int64
61   \2_LS_501_AL             1209601 non-null  int64
62   \2_LS_601_AH             1209601 non-null  int64
63   \2_LS_601_AL             1209601 non-null  int64
64   \2_LT_001_PV             1209601 non-null  float64
65   \2_LT_002_PV             1209601 non-null  float64
66   \2_MCV_007_CO            1209601 non-null  int64
67   \2_MCV_101_CO            1209601 non-null  float64
68   \2_MCV_201_CO            1209601 non-null  float64
69   \2_MCV_301_CO            1209601 non-null  float64
70   \2_MCV_401_CO            1209601 non-null  float64
71   \2_MCV_501_CO            1209601 non-null  float64
72   \2_MCV_601_CO            1209601 non-null  float64
73   \2_MV_001_STATUS         1209601 non-null  int64
74   \2_MV_002_STATUS         1209601 non-null  int64
75   \2_MV_003_STATUS         1209601 non-null  int64
76   \2_MV_004_STATUS         1209601 non-null  int64
77   \2_MV_005_STATUS         1209601 non-null  int64
78   \2_MV_006_STATUS         1209601 non-null  int64
79   \2_MV_009_STATUS         1209601 non-null  int64
80   \2_MV_101_STATUS         1209601 non-null  int64
81   \2_MV_201_STATUS         1209601 non-null  int64
82   \2_MV_301_STATUS         1209601 non-null  int64
83   \2_MV_401_STATUS         1209601 non-null  int64
84   \2_MV_501_STATUS         1209601 non-null  int64
85   \2_MV_601_STATUS         1209601 non-null  int64
86   \2_P_001_STATUS          0 non-null        float64
87   \2_P_002_STATUS          0 non-null        float64
88   \2_P_003_SPEED           1209601 non-null  float64
89   \2_P_003_STATUS          1209601 non-null  int64
90   \2_P_004_SPEED           1209601 non-null  float64
91   \2_P_004_STATUS          1209601 non-null  int64
92   \2_PIC_003_CO            1209601 non-null  float64
93   \2_PIC_003_PV            1209601 non-null  float64
94   \2_PIC_003_SP            1209601 non-null  int64
```

```
 95   \2_PIT_001_PV              1209601 non-null  float64
 96   \2_PIT_002_PV              1209601 non-null  float64
 97   \2_PIT_003_PV              1209601 non-null  float64
 98   \2_SV_101_STATUS           1209601 non-null  int64
 99   \2_SV_201_STATUS           1209601 non-null  int64
 100  \2_SV_301_STATUS           1209601 non-null  int64
 101  \2_SV_401_STATUS           1209601 non-null  int64
 102  \2_SV_501_STATUS           1209601 non-null  int64
 103  \2_SV_601_STATUS           1209601 non-null  int64
 104  \2A_AIT_001_PV             1209601 non-null  float64
 105  \2A_AIT_002_PV             1209601 non-null  float64
 106  \2A_AIT_003_PV             1209601 non-null  float64
 107  \2A_AIT_004_PV             1209601 non-null  float64
 108  \2B_AIT_001_PV             1209601 non-null  float64
 109  \2B_AIT_002_PV             1209601 non-null  float64
 110  \2B_AIT_003_PV             1209601 non-null  float64
 111  \2B_AIT_004_PV             1209581 non-null  float64
 112  \3_AIT_001_PV              1209601 non-null  float64
 113  \3_AIT_002_PV              1209595 non-null  float64
 114  \3_AIT_003_PV              1209601 non-null  float64
 115  \3_AIT_004_PV              1209595 non-null  float64
 116  \3_AIT_005_PV              1209601 non-null  float64
 117  \3_FIT_001_PV              1209601 non-null  float64
 118  \3_LS_001_AL               1209601 non-null  int64
 119  \3_LT_001_PV               1209601 non-null  float64
 120  \3_MV_001_STATUS           1209601 non-null  int64
 121  \3_MV_002_STATUS           1209601 non-null  int64
 122  \3_MV_003_STATUS           1209601 non-null  int64
 123  \3_P_001_STATUS            1209601 non-null  int64
 124  \3_P_002_STATUS            1209601 non-null  int64
 125  \3_P_003_STATUS            1209601 non-null  int64
 126  \3_P_004_STATUS            1209601 non-null  int64
 127  \LEAK_DIFF_PRESSURE        1209601 non-null  float64
 128  \PLANT_START_STOP_LOG      1209601 non-null  int64
 129  \TOTAL_CONS_REQUIRED_FLOW  1209601 non-null  float64
dtypes: float64(71), int64(57), object(2)
memory usage: 1.2+ GB
None
             Row       Date            Time  \1_AIT_001_PV  \1_AIT_002_PV  \
0              1  9/25/2017  6:00:00.000 PM        171.155       0.619473
1              2  9/25/2017  6:00:01.000 PM        171.155       0.619473
2              3  9/25/2017  6:00:02.000 PM        171.155       0.619473
3              4  9/25/2017  6:00:03.000 PM        171.155       0.607477
4              5  9/25/2017  6:00:04.000 PM        171.155       0.607477
…          …        …             …                …             …
1209596  1209597  10/9/2017  5:59:56.000 PM        164.211       0.511488
1209597  1209598  10/9/2017  5:59:57.000 PM        164.211       0.511488
1209598  1209599  10/9/2017  5:59:58.000 PM        164.211       0.511488
```

```
1209599  1209600  10/9/2017  5:59:59.000 PM        164.211       0.511488
1209600  1209601  10/9/2017  6:00:00.000 PM        164.210       0.529486

         \1_AIT_003_PV  \1_AIT_004_PV  \1_AIT_005_PV  \1_FIT_001_PV  \
0              11.5759        504.645       0.318319       0.001157
1              11.5759        504.645       0.318319       0.001157
2              11.5759        504.645       0.318319       0.001157
3              11.5725        504.673       0.318438       0.001207
4              11.5725        504.673       0.318438       0.001207
…                …              …             …              …
1209596        11.9928        482.452       0.331198       0.001182
1209597        11.9928        482.452       0.331198       0.001182
1209598        11.9928        482.452       0.331198       0.001182
1209599        11.9928        482.452       0.331198       0.001182
1209600        11.9972        482.480       0.331167       0.001273

         \1_LS_001_AL  …  \3_MV_001_STATUS  \3_MV_002_STATUS  \
0                   0  …                 1                 1
1                   0  …                 1                 1
2                   0  …                 1                 1
3                   0  …                 1                 1
4                   0  …                 1                 1
…                 …  …                 …                 …
1209596             0  …                 1                 1
1209597             0  …                 1                 1
1209598             0  …                 1                 1
1209599             0  …                 1                 1
1209600             0  …                 1                 1

         \3_MV_003_STATUS  \3_P_001_STATUS  \3_P_002_STATUS  \3_P_003_STATUS  \
0                       1                 1                 1                 1
1                       1                 1                 1                 1
2                       1                 1                 1                 1
3                       1                 1                 1                 1
4                       1                 1                 1                 1
…                     …                 …                 …                 …
1209596                 1                 1                 1                 1
1209597                 1                 1                 1                 1
1209598                 1                 1                 1                 1
1209599                 1                 1                 1                 1
1209600                 1                 1                 1                 1

         \3_P_004_STATUS  \LEAK_DIFF_PRESSURE  \PLANT_START_STOP_LOG  \
0                      1               67.9651                      1
1                      1               67.9651                      1
2                      1               67.9651                      1
3                      1               67.1948                      1
4                      1               67.1948                      1
```

```
…                    …                    …                    …
1209596                1              62.8802                    1
1209597                1              62.8802                    1
1209598                1              62.8802                    1
1209599                1              62.8802                    1
1209600                1              62.6226                    1

         \TOTAL_CONS_REQUIRED_FLOW
0                             0.68
1                             0.68
2                             0.68
3                             0.68
4                             0.68
…                              …
1209596                       0.39
1209597                       0.39
1209598                       0.39
1209599                       0.39
1209600                       0.39

[1209601 rows x 130 columns]
```

[4]:
```
print(f"Data shape of WADI_attackdata")
print(test17_df.shape)
```

```
Data shape of WADI_attackdata
(172801, 130)
```

[5]:
```
print(f"Data info of WADI_attackdata")
print(test17_df.info(verbose=True, show_counts=True))
print(test17_df)
```

```
Data info of WADI_attackdata
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 172801 entries, 0 to 172800
Data columns (total 130 columns):
 #    Column                     Non-Null Count   Dtype
---   ------                     --------------   -----
 0    Row                        172801 non-null  int64
 1    Date                       172801 non-null  object
 2    Time                       172801 non-null  object
 3    \1_AIT_001_PV              172801 non-null  float64
 4    \1_AIT_002_PV              172801 non-null  float64
 5    \1_AIT_003_PV              172801 non-null  float64
 6    \1_AIT_004_PV              172801 non-null  float64
 7    \1_AIT_005_PV              172801 non-null  float64
 8    \1_FIT_001_PV              172801 non-null  float64
 9    \1_LS_001_AL               172801 non-null  int64
```

```
10   \1_LS_002_AL             172801 non-null  int64
11   \1_LT_001_PV             172801 non-null  float64
12   \1_MV_001_STATUS         172801 non-null  int64
13   \1_MV_002_STATUS         172801 non-null  int64
14   \1_MV_003_STATUS         172801 non-null  int64
15   \1_MV_004_STATUS         172801 non-null  int64
16   \1_P_001_STATUS          172801 non-null  int64
17   \1_P_002_STATUS          172801 non-null  int64
18   \1_P_003_STATUS          172801 non-null  int64
19   \1_P_004_STATUS          172801 non-null  int64
20   \1_P_005_STATUS          172801 non-null  int64
21   \1_P_006_STATUS          172801 non-null  int64
22   \2_DPIT_001_PV           172801 non-null  float64
23   \2_FIC_101_CO            172801 non-null  float64
24   \2_FIC_101_PV            172801 non-null  float64
25   \2_FIC_101_SP            172801 non-null  float64
26   \2_FIC_201_CO            172801 non-null  float64
27   \2_FIC_201_PV            172801 non-null  float64
28   \2_FIC_201_SP            172801 non-null  float64
29   \2_FIC_301_CO            172801 non-null  float64
30   \2_FIC_301_PV            172801 non-null  float64
31   \2_FIC_301_SP            172801 non-null  float64
32   \2_FIC_401_CO            172801 non-null  float64
33   \2_FIC_401_PV            172801 non-null  float64
34   \2_FIC_401_SP            172801 non-null  float64
35   \2_FIC_501_CO            172801 non-null  float64
36   \2_FIC_501_PV            172801 non-null  float64
37   \2_FIC_501_SP            172801 non-null  float64
38   \2_FIC_601_CO            172801 non-null  float64
39   \2_FIC_601_PV            172801 non-null  float64
40   \2_FIC_601_SP            172801 non-null  float64
41   \2_FIT_001_PV            172801 non-null  float64
42   \2_FIT_002_PV            172801 non-null  float64
43   \2_FIT_003_PV            172801 non-null  float64
44   \2_FQ_101_PV             172801 non-null  float64
45   \2_FQ_201_PV             172801 non-null  float64
46   \2_FQ_301_PV             172801 non-null  float64
47   \2_FQ_401_PV             172801 non-null  float64
48   \2_FQ_501_PV             172801 non-null  float64
49   \2_FQ_601_PV             172801 non-null  float64
50   \2_LS_001_AL             0 non-null       float64
51   \2_LS_002_AL             0 non-null       float64
52   \2_LS_101_AH             172801 non-null  int64
53   \2_LS_101_AL             172801 non-null  int64
54   \2_LS_201_AH             172801 non-null  int64
55   \2_LS_201_AL             172801 non-null  int64
56   \2_LS_301_AH             172801 non-null  int64
57   \2_LS_301_AL             172801 non-null  int64
```

```
58   \2_LS_401_AH             172801 non-null  int64
59   \2_LS_401_AL             172801 non-null  int64
60   \2_LS_501_AH             172801 non-null  int64
61   \2_LS_501_AL             172801 non-null  int64
62   \2_LS_601_AH             172801 non-null  int64
63   \2_LS_601_AL             172801 non-null  int64
64   \2_LT_001_PV             172801 non-null  float64
65   \2_LT_002_PV             172801 non-null  float64
66   \2_MCV_007_CO            172801 non-null  int64
67   \2_MCV_101_CO            172801 non-null  float64
68   \2_MCV_201_CO            172801 non-null  float64
69   \2_MCV_301_CO            172801 non-null  float64
70   \2_MCV_401_CO            172801 non-null  float64
71   \2_MCV_501_CO            172801 non-null  float64
72   \2_MCV_601_CO            172801 non-null  float64
73   \2_MV_001_STATUS         172801 non-null  int64
74   \2_MV_002_STATUS         172801 non-null  int64
75   \2_MV_003_STATUS         172801 non-null  int64
76   \2_MV_004_STATUS         172801 non-null  int64
77   \2_MV_005_STATUS         172801 non-null  int64
78   \2_MV_006_STATUS         172801 non-null  int64
79   \2_MV_009_STATUS         172801 non-null  int64
80   \2_MV_101_STATUS         172801 non-null  int64
81   \2_MV_201_STATUS         172801 non-null  int64
82   \2_MV_301_STATUS         172801 non-null  int64
83   \2_MV_401_STATUS         172801 non-null  int64
84   \2_MV_501_STATUS         172801 non-null  int64
85   \2_MV_601_STATUS         172801 non-null  int64
86   \2_P_001_STATUS          0 non-null       float64
87   \2_P_002_STATUS          0 non-null       float64
88   \2_P_003_SPEED           172801 non-null  float64
89   \2_P_003_STATUS          172801 non-null  int64
90   \2_P_004_SPEED           172801 non-null  float64
91   \2_P_004_STATUS          172801 non-null  int64
92   \2_PIC_003_CO            172801 non-null  float64
93   \2_PIC_003_PV            172801 non-null  float64
94   \2_PIC_003_SP            172801 non-null  float64
95   \2_PIT_001_PV            172801 non-null  float64
96   \2_PIT_002_PV            172801 non-null  float64
97   \2_PIT_003_PV            172801 non-null  float64
98   \2_SV_101_STATUS         172801 non-null  int64
99   \2_SV_201_STATUS         172801 non-null  int64
100  \2_SV_301_STATUS         172801 non-null  int64
101  \2_SV_401_STATUS         172801 non-null  int64
102  \2_SV_501_STATUS         172801 non-null  int64
103  \2_SV_601_STATUS         172801 non-null  int64
104  \2A_AIT_001_PV           172801 non-null  float64
105  \2A_AIT_002_PV           172801 non-null  float64
```

```
 106  \2A_AIT_003_PV             172801 non-null  float64
 107  \2A_AIT_004_PV             172801 non-null  float64
 108  \2B_AIT_001_PV             172801 non-null  float64
 109  \2B_AIT_002_PV             172801 non-null  float64
 110  \2B_AIT_003_PV             172801 non-null  float64
 111  \2B_AIT_004_PV             172801 non-null  float64
 112  \3_AIT_001_PV              172801 non-null  int64
 113  \3_AIT_002_PV              172801 non-null  float64
 114  \3_AIT_003_PV              172801 non-null  float64
 115  \3_AIT_004_PV              172801 non-null  float64
 116  \3_AIT_005_PV              172801 non-null  float64
 117  \3_FIT_001_PV              172801 non-null  float64
 118  \3_LS_001_AL               172801 non-null  int64
 119  \3_LT_001_PV               172801 non-null  float64
 120  \3_MV_001_STATUS           172801 non-null  int64
 121  \3_MV_002_STATUS           172801 non-null  int64
 122  \3_MV_003_STATUS           172801 non-null  int64
 123  \3_P_001_STATUS            172801 non-null  int64
 124  \3_P_002_STATUS            172801 non-null  int64
 125  \3_P_003_STATUS            172801 non-null  int64
 126  \3_P_004_STATUS            172801 non-null  int64
 127  \LEAK_DIFF_PRESSURE        172801 non-null  float64
 128  \PLANT_START_STOP_LOG      172801 non-null  int64
 129  \TOTAL_CONS_REQUIRED_FLOW  172801 non-null  float64
dtypes: float64(71), int64(57), object(2)
memory usage: 171.4+ MB
None
           Row        Date            Time  \1_AIT_001_PV  \1_AIT_002_PV  \
0            1   10/9/2017  6:00:00.000 PM        164.210       0.529486
1            2   10/9/2017  6:00:01.000 PM        164.210       0.529486
2            3   10/9/2017  6:00:02.000 PM        164.210       0.529486
3            4   10/9/2017  6:00:03.000 PM        164.210       0.529486
4            5   10/9/2017  6:00:04.000 PM        164.210       0.529486
…          …          …               …              …             …
172796  172797  10/11/2017  5:59:56.000 PM        172.959       0.547483
172797  172798  10/11/2017  5:59:57.000 PM        172.959       0.547483
172798  172799  10/11/2017  5:59:58.000 PM        172.915       0.583479
172799  172800  10/11/2017  5:59:59.000 PM        172.915       0.583479
172800  172801  10/11/2017  6:00:00.000 PM        172.915       0.583479

        \1_AIT_003_PV  \1_AIT_004_PV  \1_AIT_005_PV  \1_FIT_001_PV  \
0             11.9972        482.480       0.331167       0.001273
1             11.9972        482.480       0.331167       0.001273
2             11.9972        482.480       0.331167       0.001273
3             11.9972        482.480       0.331167       0.001273
4             11.9972        482.480       0.331167       0.001273
…               …              …              …              …
172796        11.9184        466.034       0.318217       0.001222
```

```
172797        11.9184        466.034       0.318217       0.001222
172798        11.9211        466.051       0.318317       0.001260
172799        11.9211        466.051       0.318317       0.001260
172800        11.9211        466.051       0.318317       0.001260

        \1_LS_001_AL  …  \3_MV_001_STATUS  \3_MV_002_STATUS  \
0                  0  …                 1                 1
1                  0  …                 1                 1
2                  0  …                 1                 1
3                  0  …                 1                 1
4                  0  …                 1                 1
…                …  …                 …                 …
172796             0  …                 1                 1
172797             0  …                 1                 1
172798             0  …                 1                 1
172799             0  …                 1                 1
172800             0  …                 1                 1

        \3_MV_003_STATUS  \3_P_001_STATUS  \3_P_002_STATUS  \3_P_003_STATUS  \
0                      1                 1                 1                 1
1                      1                 1                 1                 1
2                      1                 1                 1                 1
3                      1                 1                 1                 1
4                      1                 1                 1                 1
…                    …                 …                 …                 …
172796                 1                 1                 1                 1
172797                 1                 1                 1                 1
172798                 1                 1                 1                 1
172799                 1                 1                 1                 1
172800                 1                 1                 1                 1

        \3_P_004_STATUS  \LEAK_DIFF_PRESSURE  \PLANT_START_STOP_LOG  \
0                     1              62.6226                      1
1                     1              62.6226                      1
2                     1              62.6226                      1
3                     1              62.6226                      1
4                     1              62.6226                      1
…                   …                    …                    …
172796                1              59.3546                      1
172797                1              59.3546                      1
172798                1              58.8102                      1
172799                1              58.8102                      1
172800                1              58.8102                      1

        \TOTAL_CONS_REQUIRED_FLOW
0                            0.39
1                            0.39
2                            0.39
```

```
3                              0.39
4                              0.39
…                              …
172796                         0.00
172797                         0.00
172798                         0.00
172799                         0.00
172800                         0.00

[172801 rows x 130 columns]
```

1.3.3 Data Cleaning

[6]:
```
print(f"Perform Data Cleaning")
# Remove duplicates (if any)
train17_df.drop_duplicates(inplace=True)
test17_df.drop_duplicates(inplace=True)
```

```
Perform Data Cleaning
```

[7]:
```
# Find columns where the count of non-null values is equal to 0
cols_to_drop = train17_df.columns[train17_df.notnull().sum() == 0]

# Drop those columns from the DataFrame
train17_df = train17_df.drop(columns=cols_to_drop)
```

[8]:
```
# Find columns where the count of non-null values is equal to 0
cols_to_drop = test17_df.columns[test17_df.notnull().sum() == 0]

# Drop those columns from the DataFrame
test17_df = test17_df.drop(columns=cols_to_drop)
```

[9]:
```
# Check for null values
pd.set_option('display.max_rows', None)
train17_df.isnull().sum()
pd.reset_option('display.max_rows')
```

[10]:
```
# Check for null values
pd.set_option('display.max_rows', None)
test17_df.isnull().sum()
pd.reset_option('display.max_rows')
```

[11]:
```
# Remove null values
train17_df = train17_df.ffill().bfill()
test17_df = test17_df.ffill().bfill()
```

[12]:
```
# Check for null values after remoe
train17_df.isnull().sum().sum()
test17_df.isnull().sum().sum()
```

[12]:
```
np.int64(0)
```

1.4 Classify data columns into Meta Group, Sensor Group or Actuator Group

[13]:
```
print(f"Classify the data columns into MetaGroup, Sensor Group and Actuator␣
 ↪Group - 14days")
# ========================================
# EXPLICIT COLUMN GROUPING
# ========================================
all_cols = train17_df.columns.tolist()

metadata_cols = []
actuator_ctrl_cols = []
sensor_cols = []

for col in all_cols:
    col_upper = col.upper()

    # Identify Metadata columns
    if any(k in col_upper for k in ["ROW", "DATE", "TIME"]):
        metadata_cols.append(col)

    # Identify Continuous Sensor Process Values (_PV and direct flow/pressure␣
 ↪metrics)
    elif "_PV" in col_upper or any(
        k in col_upper for k in ["LEAK_DIFF_PRESSURE",␣
 ↪"TOTAL_CONS_REQUIRED_FLOW"]
    ):
        sensor_cols.append(col)

    # Identify Actuators, Alarms, Controller Outputs, and Setpoints␣
 ↪(non-continuous sensors)
    elif any(
        k in col_upper
        for k in [
            "STATUS",
            "_AL",
            "_AH",
            "_CO",
            "_SP",
            "_SPEED",
            "PLANT_START_STOP_LOG",
```

```
        ]
    ):
        actuator_ctrl_cols.append(col)


    else:
        # Fallback safety: treat anything else as actuator/control if not␣
 ↪explicitly a sensor
        actuator_ctrl_cols.append(col)

print(f"\nGroup Breakdown:")
print(f" - Metadata Columns: {len(metadata_cols)}")
print(f" - Actuator Columns: {len(actuator_ctrl_cols)}")
print(f" - Sensor Columns: {len(sensor_cols)}")
```

```
Classify the data columns into MetaGroup, Sensor Group and Actuator Group -
14days

Group Breakdown:
 - Metadata Columns: 3
 - Actuator Columns: 77
 - Sensor Columns: 46
```

[14]:
```
print(f"Classify the data columns into MetaGroup, Sensor Group and Actuator␣
 ↪Group - attackdata")
# ========================================
# EXPLICIT COLUMN GROUPING
# ========================================
tall_cols = test17_df.columns.tolist()

tmetadata_cols = []
tactuator_ctrl_cols = []
tsensor_cols = []

for col in tall_cols:
    col_upper = col.upper()

    # Identify Metadata columns
    if any(k in col_upper for k in ["ROW", "DATE", "TIME"]):
        tmetadata_cols.append(col)

    # Identify Continuous Sensor Process Values (_PV and direct flow/pressure␣
 ↪metrics)
    elif "_PV" in col_upper or any(
        k in col_upper for k in ["LEAK_DIFF_PRESSURE",␣
 ↪"TOTAL_CONS_REQUIRED_FLOW"]
    ):
```

```
        tsensor_cols.append(col)

    # Identify Actuators, Alarms, Controller Outputs, and Setpoints␣
 ↪(non-continuous sensors)
    elif any(
        k in col_upper
        for k in [
            "STATUS",
            "_AL",
            "_AH",
            "_CO",
            "_SP",
            "_SPEED",
            "PLANT_START_STOP_LOG",
        ]
    ):
        tactuator_ctrl_cols.append(col)


    else:
        # Fallback safety: treat anything else as actuator/control if not␣
 ↪explicitly a sensor
        tactuator_ctrl_cols.append(col)

print(f"\nGroup Breakdown:")
print(f" - Metadata Columns: {len(tmetadata_cols)}")
print(f" - Actuator Columns: {len(tactuator_ctrl_cols)}")
print(f" - Sensor Columns: {len(tsensor_cols)}")
```

```
Classify the data columns into MetaGroup, Sensor Group and Actuator Group -
attackdata

Group Breakdown:
 - Metadata Columns: 3
 - Actuator Columns: 77
 - Sensor Columns: 46
```

1.5 Add new Column Attack Label and Split Attack data from attackdata dataset

[15]:
```
print(f"Add Attack Label and classify attack label based on specified time␣
 ↪ranges - attackdata")
# Clean column names in case of whitespace artifacts
test17_df.columns = test17_df.columns.str.strip()

# ========================================
# PARSE TIMESTAMP USING 'Date' AND 'Time' COLUMNS
```

```
# ========================================
# Create a temporary datetime series directly from 'DATE' and 'TIME'
# without adding a permanent 'DateTime' column to test17_df
temp_datetime = pd.to_datetime(
    test17_df["Date"].astype(str).str.strip()
    + " "
    + test17_df["Time"].astype(str).str.strip(),
    format='mixed',
    errors="coerce"
)

# ========================================
# DEFINE ATTACK TIME RANGES
# ========================================
attack_ranges = [
    ("09/10/2017 19:25:00", "09/10/2017 19:50:16"),
    ("10/10/2017 10:24:10", "10/10/2017 10:34:00"),
    ("10/10/2017 10:55:00", "10/10/2017 11:24:00"),
    ("10/10/2017 11:07:46", "10/10/2017 11:12:15"),
    ("10/10/2017 11:30:40", "10/10/2017 11:44:50"),
    ("10/10/2017 13:39:30", "10/10/2017 13:50:40"),
    ("10/10/2017 14:48:17", "10/10/2017 14:59:55"),
    ("10/10/2017 14:53:44", "10/10/2017 15:00:32"),
    ("10/10/2017 17:40:00", "10/10/2017 17:49:40"),
    ("11/10/2017 10:55:00", "11/10/2017 10:56:27"),
    ("11/10/2017 11:17:54", "11/10/2017 11:31:20"),
    ("11/10/2017 11:36:31", "11/10/2017 11:47:00"),
    ("11/10/2017 11:59:00", "11/10/2017 12:05:00"),
    ("11/10/2017 12:07:30", "11/10/2017 12:10:52"),
    ("11/10/2017 12:16:00", "11/10/2017 12:25:36"),
    ("11/10/2017 15:26:30", "11/10/2017 15:37:00"),
]

# ========================================
# INITIALIZE AND POPULATE ATTACK LABELS
# ========================================
# Initialize attack label column to 1 as default
test17_df["ATTACK_LABEL"] = 1

# Iterate through each attack range and mark matching rows as -1
for start_str, end_str in attack_ranges:
    start_dt = pd.to_datetime(start_str, format="%d/%m/%Y %H:%M:%S")
    end_dt = pd.to_datetime(end_str, format="%d/%m/%Y %H:%M:%S")

    mask = (temp_datetime >= start_dt) & (temp_datetime <= end_dt)
    test17_df.loc[mask, "ATTACK_LABEL"] = -1
```

```
# ========================================
# VERIFICATION
# ========================================
print("\nLabel distribution summary:")
print(test17_df["ATTACK_LABEL"].value_counts())
print(f"\nFinal DataFrame shape: {test17_df.shape}")
print(f"Last column name: {test17_df.columns[-1]}")
```

```
Add Attack Label and classify attack label based on specified time ranges -
attackdata

Label distribution summary:
ATTACK_LABEL
 1    162816
-1      9985
Name: count, dtype: int64

Final DataFrame shape: (172801, 127)
Last column name: ATTACK_LABEL
```

[16]:
```
print(f"Data info of attackdata after adding new Attack Label Column")
print(test17_df.info(verbose=True, show_counts=True))
print(test17_df)
```

```
Data info of attackdata after adding new Attack Label Column
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 172801 entries, 0 to 172800
Data columns (total 127 columns):
 #    Column                      Non-Null Count   Dtype
---   ------                      --------------   -----
 0    Row                         172801 non-null  int64
 1    Date                        172801 non-null  object
 2    Time                        172801 non-null  object
 3    \1_AIT_001_PV               172801 non-null  float64
 4    \1_AIT_002_PV               172801 non-null  float64
 5    \1_AIT_003_PV               172801 non-null  float64
 6    \1_AIT_004_PV               172801 non-null  float64
 7    \1_AIT_005_PV               172801 non-null  float64
 8    \1_FIT_001_PV               172801 non-null  float64
 9    \1_LS_001_AL                172801 non-null  int64
 10   \1_LS_002_AL                172801 non-null  int64
 11   \1_LT_001_PV                172801 non-null  float64
 12   \1_MV_001_STATUS            172801 non-null  int64
 13   \1_MV_002_STATUS            172801 non-null  int64
 14   \1_MV_003_STATUS            172801 non-null  int64
 15   \1_MV_004_STATUS            172801 non-null  int64
 16   \1_P_001_STATUS             172801 non-null  int64
 17   \1_P_002_STATUS             172801 non-null  int64
```

```
18   \1_P_003_STATUS           172801 non-null  int64
19   \1_P_004_STATUS           172801 non-null  int64
20   \1_P_005_STATUS           172801 non-null  int64
21   \1_P_006_STATUS           172801 non-null  int64
22   \2_DPIT_001_PV            172801 non-null  float64
23   \2_FIC_101_CO             172801 non-null  float64
24   \2_FIC_101_PV             172801 non-null  float64
25   \2_FIC_101_SP             172801 non-null  float64
26   \2_FIC_201_CO             172801 non-null  float64
27   \2_FIC_201_PV             172801 non-null  float64
28   \2_FIC_201_SP             172801 non-null  float64
29   \2_FIC_301_CO             172801 non-null  float64
30   \2_FIC_301_PV             172801 non-null  float64
31   \2_FIC_301_SP             172801 non-null  float64
32   \2_FIC_401_CO             172801 non-null  float64
33   \2_FIC_401_PV             172801 non-null  float64
34   \2_FIC_401_SP             172801 non-null  float64
35   \2_FIC_501_CO             172801 non-null  float64
36   \2_FIC_501_PV             172801 non-null  float64
37   \2_FIC_501_SP             172801 non-null  float64
38   \2_FIC_601_CO             172801 non-null  float64
39   \2_FIC_601_PV             172801 non-null  float64
40   \2_FIC_601_SP             172801 non-null  float64
41   \2_FIT_001_PV             172801 non-null  float64
42   \2_FIT_002_PV             172801 non-null  float64
43   \2_FIT_003_PV             172801 non-null  float64
44   \2_FQ_101_PV              172801 non-null  float64
45   \2_FQ_201_PV              172801 non-null  float64
46   \2_FQ_301_PV              172801 non-null  float64
47   \2_FQ_401_PV              172801 non-null  float64
48   \2_FQ_501_PV              172801 non-null  float64
49   \2_FQ_601_PV              172801 non-null  float64
50   \2_LS_101_AH              172801 non-null  int64
51   \2_LS_101_AL              172801 non-null  int64
52   \2_LS_201_AH              172801 non-null  int64
53   \2_LS_201_AL              172801 non-null  int64
54   \2_LS_301_AH              172801 non-null  int64
55   \2_LS_301_AL              172801 non-null  int64
56   \2_LS_401_AH              172801 non-null  int64
57   \2_LS_401_AL              172801 non-null  int64
58   \2_LS_501_AH              172801 non-null  int64
59   \2_LS_501_AL              172801 non-null  int64
60   \2_LS_601_AH              172801 non-null  int64
61   \2_LS_601_AL              172801 non-null  int64
62   \2_LT_001_PV              172801 non-null  float64
63   \2_LT_002_PV              172801 non-null  float64
64   \2_MCV_007_CO             172801 non-null  int64
65   \2_MCV_101_CO             172801 non-null  float64
```

```
66   \2_MCV_201_CO             172801 non-null  float64
67   \2_MCV_301_CO             172801 non-null  float64
68   \2_MCV_401_CO             172801 non-null  float64
69   \2_MCV_501_CO             172801 non-null  float64
70   \2_MCV_601_CO             172801 non-null  float64
71   \2_MV_001_STATUS          172801 non-null  int64
72   \2_MV_002_STATUS          172801 non-null  int64
73   \2_MV_003_STATUS          172801 non-null  int64
74   \2_MV_004_STATUS          172801 non-null  int64
75   \2_MV_005_STATUS          172801 non-null  int64
76   \2_MV_006_STATUS          172801 non-null  int64
77   \2_MV_009_STATUS          172801 non-null  int64
78   \2_MV_101_STATUS          172801 non-null  int64
79   \2_MV_201_STATUS          172801 non-null  int64
80   \2_MV_301_STATUS          172801 non-null  int64
81   \2_MV_401_STATUS          172801 non-null  int64
82   \2_MV_501_STATUS          172801 non-null  int64
83   \2_MV_601_STATUS          172801 non-null  int64
84   \2_P_003_SPEED            172801 non-null  float64
85   \2_P_003_STATUS           172801 non-null  int64
86   \2_P_004_SPEED            172801 non-null  float64
87   \2_P_004_STATUS           172801 non-null  int64
88   \2_PIC_003_CO             172801 non-null  float64
89   \2_PIC_003_PV             172801 non-null  float64
90   \2_PIC_003_SP             172801 non-null  float64
91   \2_PIT_001_PV             172801 non-null  float64
92   \2_PIT_002_PV             172801 non-null  float64
93   \2_PIT_003_PV             172801 non-null  float64
94   \2_SV_101_STATUS          172801 non-null  int64
95   \2_SV_201_STATUS          172801 non-null  int64
96   \2_SV_301_STATUS          172801 non-null  int64
97   \2_SV_401_STATUS          172801 non-null  int64
98   \2_SV_501_STATUS          172801 non-null  int64
99   \2_SV_601_STATUS          172801 non-null  int64
100  \2A_AIT_001_PV            172801 non-null  float64
101  \2A_AIT_002_PV            172801 non-null  float64
102  \2A_AIT_003_PV            172801 non-null  float64
103  \2A_AIT_004_PV            172801 non-null  float64
104  \2B_AIT_001_PV            172801 non-null  float64
105  \2B_AIT_002_PV            172801 non-null  float64
106  \2B_AIT_003_PV            172801 non-null  float64
107  \2B_AIT_004_PV            172801 non-null  float64
108  \3_AIT_001_PV             172801 non-null  int64
109  \3_AIT_002_PV             172801 non-null  float64
110  \3_AIT_003_PV             172801 non-null  float64
111  \3_AIT_004_PV             172801 non-null  float64
112  \3_AIT_005_PV             172801 non-null  float64
113  \3_FIT_001_PV             172801 non-null  float64
```

```
 114  \3_LS_001_AL               172801 non-null  int64
 115  \3_LT_001_PV               172801 non-null  float64
 116  \3_MV_001_STATUS           172801 non-null  int64
 117  \3_MV_002_STATUS           172801 non-null  int64
 118  \3_MV_003_STATUS           172801 non-null  int64
 119  \3_P_001_STATUS            172801 non-null  int64
 120  \3_P_002_STATUS            172801 non-null  int64
 121  \3_P_003_STATUS            172801 non-null  int64
 122  \3_P_004_STATUS            172801 non-null  int64
 123  \LEAK_DIFF_PRESSURE        172801 non-null  float64
 124  \PLANT_START_STOP_LOG      172801 non-null  int64
 125  \TOTAL_CONS_REQUIRED_FLOW  172801 non-null  float64
 126  ATTACK_LABEL               172801 non-null  int64
dtypes: float64(67), int64(58), object(2)
memory usage: 167.4+ MB
None
           Row        Date             Time  \1_AIT_001_PV  \1_AIT_002_PV  \
0            1   10/9/2017  6:00:00.000 PM        164.210       0.529486
1            2   10/9/2017  6:00:01.000 PM        164.210       0.529486
2            3   10/9/2017  6:00:02.000 PM        164.210       0.529486
3            4   10/9/2017  6:00:03.000 PM        164.210       0.529486
4            5   10/9/2017  6:00:04.000 PM        164.210       0.529486
…          …          …                …              …             …
172796  172797  10/11/2017  5:59:56.000 PM        172.959       0.547483
172797  172798  10/11/2017  5:59:57.000 PM        172.959       0.547483
172798  172799  10/11/2017  5:59:58.000 PM        172.915       0.583479
172799  172800  10/11/2017  5:59:59.000 PM        172.915       0.583479
172800  172801  10/11/2017  6:00:00.000 PM        172.915       0.583479

        \1_AIT_003_PV  \1_AIT_004_PV  \1_AIT_005_PV  \1_FIT_001_PV  \
0             11.9972        482.480       0.331167       0.001273
1             11.9972        482.480       0.331167       0.001273
2             11.9972        482.480       0.331167       0.001273
3             11.9972        482.480       0.331167       0.001273
4             11.9972        482.480       0.331167       0.001273
…               …              …              …              …
172796        11.9184        466.034       0.318217       0.001222
172797        11.9184        466.034       0.318217       0.001222
172798        11.9211        466.051       0.318317       0.001260
172799        11.9211        466.051       0.318317       0.001260
172800        11.9211        466.051       0.318317       0.001260

        \1_LS_001_AL  …  \3_MV_002_STATUS  \3_MV_003_STATUS  \
0                  0  …                 1                 1
1                  0  …                 1                 1
2                  0  …                 1                 1
3                  0  …                 1                 1
4                  0  …                 1                 1
```

```
…              …  …                …                …
172796             0  …                1                1
172797             0  …                1                1
172798             0  …                1                1
172799             0  …                1                1
172800             0  …                1                1

        \3_P_001_STATUS  \3_P_002_STATUS  \3_P_003_STATUS  \3_P_004_STATUS  \
0                     1                1                1                1
1                     1                1                1                1
2                     1                1                1                1
3                     1                1                1                1
4                     1                1                1                1
…                   …                …                …                …
172796                1                1                1                1
172797                1                1                1                1
172798                1                1                1                1
172799                1                1                1                1
172800                1                1                1                1

        \LEAK_DIFF_PRESSURE  \PLANT_START_STOP_LOG  \TOTAL_CONS_REQUIRED_FLOW  \
0                   62.6226                      1                       0.39
1                   62.6226                      1                       0.39
2                   62.6226                      1                       0.39
3                   62.6226                      1                       0.39
4                   62.6226                      1                       0.39
…                     …                        …                         …
172796              59.3546                      1                       0.00
172797              59.3546                      1                       0.00
172798              58.8102                      1                       0.00
172799              58.8102                      1                       0.00
172800              58.8102                      1                       0.00

        ATTACK_LABEL
0                  1
1                  1
2                  1
3                  1
4                  1
…                …
172796             1
172797             1
172798             1
172799             1
172800             1

[172801 rows x 127 columns]
```

[17]:
```
print(f"Separate the attackdata into 2 datasets: 1 with normal data and 1 with␣
 ↪attack data")
# ========================================
# SEPARATE DATASET BY ATTACK LABEL
# ========================================

# Filter rows where ATTACK_LABEL is 1 (Normal data)
test17_normal_df = test17_df[test17_df["ATTACK_LABEL"] == 1].copy()

# Filter rows where ATTACK_LABEL is -1 (Attack data)
test17_attack_df = test17_df[test17_df["ATTACK_LABEL"] == -1].copy()

# ========================================
# VERIFICATION
# ========================================
print(f"Original DataFrame shape: {test17_df.shape}")
print(f"Normal DataFrame shape (`test17_normal_df`): {test17_normal_df.shape}")
print(f"Attack DataFrame shape (`test17_attack_df`): {test17_attack_df.shape}")
```

```
Separate the attackdata into 2 datasets: 1 with normal data and 1 with attack
data
Original DataFrame shape: (172801, 127)
Normal DataFrame shape (`test17_normal_df`): (162816, 127)
Attack DataFrame shape (`test17_attack_df`): (9985, 127)
```

Task 2a Compute Spearman Distance

[18]:
```
print(f"Task 2a Compute Spearman Distance")
# ========================================
# PREPARE THE THREE DIFFERENTIATED DATASETS
# ========================================
# Dataset 1: Baseline Normal (train17_df)
df_train_sensors = train17_df[sensor_cols].ffill().bfill().fillna(0)

# Dataset 2: Test Data WITHOUT Attacks (test17_normal_df)
df_test_normal_sensors = test17_normal_df[sensor_cols].ffill().bfill().fillna(0)

# Dataset 3: Test Data CONTAINING Attacks (test17_attack_df)
df_test_attack_sensors = test17_attack_df[sensor_cols].ffill().bfill().fillna(0)


# ========================================
# CUSTOM SPEARMAN IMPLEMENTATION (FROM SCRATCH)
# ========================================
def compute_rank(arr):
    """Computes average ranks for ties (equivalent to scipy.stats.rankdata)."""
    sort_idx = np.argsort(arr)
    rank = np.empty_like(sort_idx, dtype=float)
```

```
    n = len(arr)
    i = 0
    while i < n:
        j = i
        while j < n - 1 and arr[sort_idx[j]] == arr[sort_idx[j + 1]]:
            j += 1
        # Assign average rank for tied values
        avg_rank = (i + j) / 2.0 + 1
        for k in range(i, j + 1):
            rank[sort_idx[k]] = avg_rank
        i = j + 1
    return rank


def custom_spearman_correlation_matrix(df):
    """Computes the Spearman rank correlation matrix for all columns

    in a DataFrame from scratch using only NumPy.
    """
    data = df.to_numpy()
    n_rows, n_cols = data.shape

    # Step 1: Convert each column to its ranks
    ranked_data = np.empty_like(data, dtype=float)
    for j in range(n_cols):
        ranked_data[:, j] = compute_rank(data[:, j])

    # Step 2: Compute Pearson correlation on the ranked data
    # Mean-center the ranks
    means = np.mean(ranked_data, axis=0)
    centered = ranked_data - means

    # Numerator: Covariance matrix of ranks
    cov_matrix = np.dot(centered.T, centered) / (n_rows - 1)

    # Denominator: Standard deviations of ranks
    std_devs = np.std(ranked_data, axis=0, ddof=1)
    outer_std = np.outer(std_devs, std_devs)

    # Avoid division by zero for constant columns
    outer_std[outer_std == 0] = 1.0

    spearman_corr = cov_matrix / outer_std
    # Clip to handle minor floating-point inaccuracies outside [-1, 1]
    spearman_corr = np.clip(spearman_corr, -1.0, 1.0)

    return spearman_corr
```

```python
# ========================================
# COMPUTE SPEARMAN DISTANCE MATRICES
# ========================================
print("\nComputing Spearman correlation and distance matrices")
print(" - Processing Baseline Normal (train17_df-WADI_14days)...")
corr_train = custom_spearman_correlation_matrix(df_train_sensors)

print(" - Processing Test Normal (test17_normal_df-WADI_attackdata)...")
corr_test_normal = custom_spearman_correlation_matrix(df_test_normal_sensors)

print(" - Processing Test Attack (test17_attack_df-WADI_attackdata)...")
corr_test_attack = custom_spearman_correlation_matrix(df_test_attack_sensors)

# Spearman Distance is defined as: Distance = 1 - Correlation
dist_train = 1.0 - corr_train
dist_test_normal = 1.0 - corr_test_normal
dist_test_attack = 1.0 - corr_test_attack


# ========================================
# SUMMARIZE & COMPARE DISTANCE STRUCTURES
# ========================================
print("\n" + "=" * 50)
print("TASK 2a: SPEARMAN DISTANCE ANALYSIS SUMMARY")
print("=" * 50)
print(f"Average Spearman Distance (Baseline Normal-WADI_14days):    {np.
 ↪mean(dist_train):.4f}")
print(f"Average Spearman Distance (Test Normal-WADI_attackdata):    {np.
 ↪mean(dist_test_normal):.4f}")
print(f"Average Spearman Distance (Test Attack-WADI_attackdata):    {np.
 ↪mean(dist_test_attack):.4f}")
```

```
Task 2a Compute Spearman Distance

Computing Spearman correlation and distance matrices
 - Processing Baseline Normal (train17_df-WADI_14days)…
 - Processing Test Normal (test17_normal_df-WADI_attackdata)…
 - Processing Test Attack (test17_attack_df-WADI_attackdata)…

==================================================
TASK 2a: SPEARMAN DISTANCE ANALYSIS SUMMARY
==================================================
Average Spearman Distance (Baseline Normal-WADI_14days):    0.9650
Average Spearman Distance (Test Normal-WADI_attackdata):    0.9687
Average Spearman Distance (Test Attack-WADI_attackdata):    0.9612
```

1.6 Task 2b Create a Histogram

[19]:
```
print(f"Task 2b Create Historgram")

# ========================================
# EXTRACT PAIRWISE SPEARMAN DISTANCES
# ========================================
# Extract off-diagonal elements from the distance matrices to avoid␣
 ↪self-correlations (diagonal = 0)
n_sensors = corr_test_normal.shape[0]
triu_indices = np.triu_indices(n_sensors, k=1)

normal_distances = (1.0 - corr_test_normal)[triu_indices]
attack_distances = (1.0 - corr_test_attack)[triu_indices]

# ========================================
# HISTOGRAM COMPUTATION
# ========================================
n_bins = 30
combined_data = np.concatenate([normal_distances, attack_distances])
min_val, max_val = np.min(combined_data), np.max(combined_data)

# Avoid division by zero if all values are identical
if min_val == max_val:
    max_val += 1e-5

bin_width = (max_val - min_val) / n_bins
bin_edges = [min_val + (i * bin_width) for i in range(n_bins + 1)]

def compute_manual_histogram(data, edges, width):
    """Computes bin frequencies manually using basic logic without libraries."""
    n_b = len(edges) - 1
    counts = [0] * n_b
    for val in data:
        # Determine the bin index mathematically
        idx = int((val - edges[0]) / width)
        if idx >= n_b:
            idx = n_b - 1
        elif idx < 0:
            idx = 0
        counts[idx] += 1
    return counts

# Compute frequencies for both distributions
normal_counts = compute_manual_histogram(normal_distances, bin_edges, bin_width)
attack_counts = compute_manual_histogram(attack_distances, bin_edges, bin_width)
```

```
# Calculate bin centers for rendering bars
bin_centers = [(bin_edges[i] + bin_edges[i + 1]) / 2.0 for i in range(n_bins)]

# ==========================================
# PLOT AND DISPLAY THE SINGLE HISTOGRAM
# ==========================================
plt.figure(figsize=(10, 6))

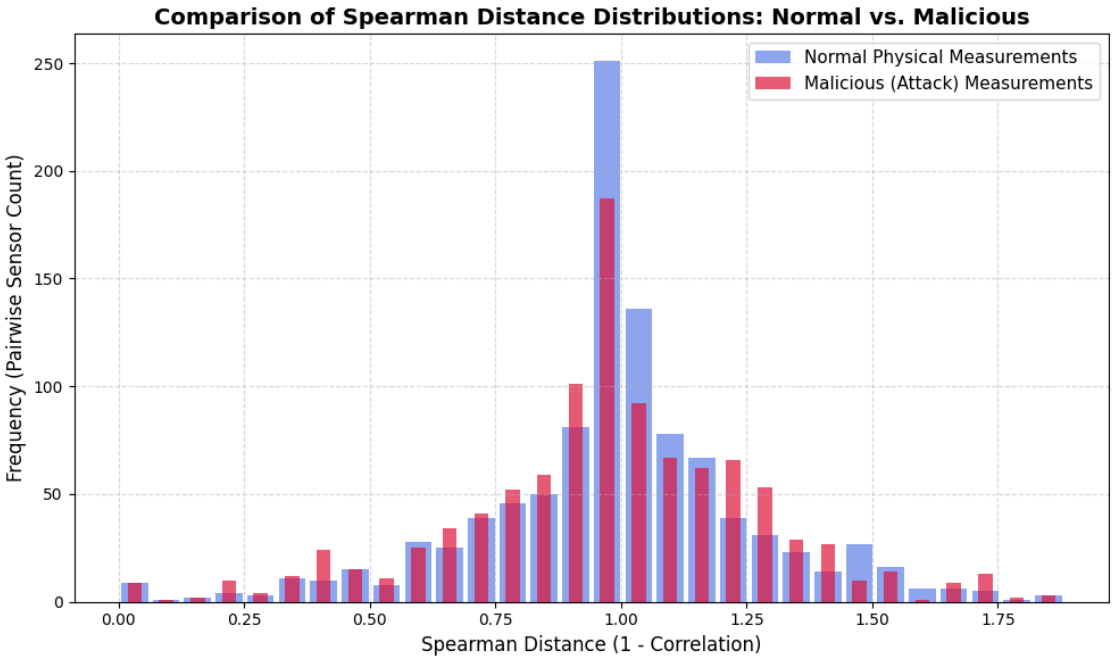
# Plot normal and malicious distributions on the same axes with transparency
plt.bar(
    bin_centers,
    normal_counts,
    width=bin_width * 0.85,
    alpha=0.6,
    color='royalblue',
    label='Normal Physical Measurements'
)
plt.bar(
    bin_centers,
    attack_counts,
    width=bin_width * 0.45,
    alpha=0.7,
    color='crimson',
    label='Malicious (Attack) Measurements'
)

plt.xlabel('Spearman Distance (1 - Correlation)', fontsize=12)
plt.ylabel('Frequency (Pairwise Sensor Count)', fontsize=12)
plt.title('Comparison of Spearman Distance Distributions: Normal vs.␣
 ↪Malicious', fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()

# Display the plot
plt.savefig('T2_historgam.png')
plt.close
```

Task 2b Create Historgram

[19]: <function matplotlib.pyplot.close(fig: "None | int | str | Figure | Literal['all']" = None) -> 'None'>


Comparison of Spearman Distance Distributions: Normal vs. Malicious
Normal Physical Measurements
Malicious (Attack) Measurements
Frequency (Pairwise Sensor Count)
Spearman Distance (1 - Correlation)
0
50
100
150
200
250
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75